Attack Surface Mapper: A Rule-Based and Optionally ML-Augmented Static Analysis Tool for Android Application Security Assessment

Noussair Bouanani[a], Amal Sab[a], Najwa Saber[a], Hiba Sidinou[a], Mohammed Lachgar[a]

[a]*Ecole nationale des sciences appliquees Marrakech, Genie cyber-defense et systemes de telecommunication embarques, Marrakech, Maroc*

Abstract

Mobile application security is a growing concern, as Android applications routinely expose sensitive components through misconfigured `AndroidManifest` files. This paper presents *Attack Surface Mapper*, an open-source Python tool that automates the static analysis of Android manifests to identify exported components, dangerous permission patterns, and configuration vulnerabilities aligned with OWASP MASTG guidelines. The tool is built around a deterministic, rule-based engine that maps findings to CWE identifiers (CWE-926, CWE-489, CWE-530, CWE-862) and computes a weighted risk score (0–100) decomposed into three categories: Exported Components, Missing Permissions, and Dangerous Permissions. An optional scikit-learn multinomial logistic regression classifier, trained on the Kaggle Android Permissions dataset (29 616 samples, 87 binary permission features), supplements the rule-based score with a binary high/low-risk prediction when its confidence exceeds a 0.80 threshold; otherwise the system falls back to the rule-based result exclusively. Results are exported as self-contained HTML reports and multi-format attack surface graphs (Mermaid, GraphML, JSON). The tool was evaluated on four open-source Android benchmarks—OWASP UnCrackable Level 2, DIVA APK, InsecureBankv2, and OWASP GoatDroid— achieving a precision of 0.91, a recall of 0.88, and an F_1 score of 0.89 on manifest-level vulnerability detection across 27 ground-truth findings. These results demonstrate that the tool offers security practitioners and mobile developers a practical, reproducible interface for continuous security assessment.

Keywords: Android security, attack surface analysis, static analysis, AndroidManifest, OWASP MASTG, mobile security, hybrid risk scoring

Required Metadata

Nr.	Code metadata description	Information
C1	Current code version	v1.1.0
C2	Permanent link to code/repository used for this code version	https://github.com/NoussairBN/attack-surface-mapper
C3	Permanent link to reproducible capsule	No executable capsule is currently hosted. The README provides step-by-step reproduction instructions: README.md. A Zenodo archive is planned for the camera-ready version.
C4	Legal Code License	MIT License
C5	Code versioning system used	Git
C6	Software code languages, tools, and services used	Python 3.x, lxml, androguard, jinja2, pyyaml, click, scikit-learn 1.4, numpy, pytest
C7	Compilation requirements, operating environments & dependencies	Python 3.8+, pip, apktool (optional for APK decompilation), Windows/-macOS/Linux. Install: `pip install -r requirements.txt`
C8	Link to developer documentation/manual	README.md on GitHub
C9	Support email for questions	m.lachgar@uca.ac.ma

1. Motivation and Significance

Android is the dominant mobile operating system worldwide, deployed on over 3.9 billion active devices as of 2024 [1]. Its open component model—which allows applications to declare activities, services, broadcast receivers, and content providers accessible to other applications via `Intent`—is a well-documented source of security vulnerabilities [16, 17]. Empirical studies on large corpora have consistently shown that a significant fraction of published applications export components without adequate permission guards. Felt et al. [16] found that over one-third of Android applications in their 2011 corpus were *overprivileged*, a pattern still observed in more recent analyses [18, 19]. These misconfigurations map directly to documented attack classes: component hijacking, intent spoofing, and unauthorised content-provider access [20].

The OWASP Mobile Application Security Verification Standard (MASVS) [3] and the Mobile Application Security Testing Guide (MASTG) [2] formalise the corresponding test cases: MASTG-TEST-0024 (Testing for Vulnerable Implementation of PendingIntent), MASTG-TEST-0025 (Testing for Implicit Intents), and MASTG-TEST-0039 (Testing Backup Flag). At the Common Weakness Enumeration level, the relevant identifiers are CWE-926 (Improper Export of Android Application Components), CWE-489 (Active Debug Code, triggered by `android:debuggable="true"`), CWE-530 (Exposure of Backup File to an Unauthorised Control Sphere, triggered by `android:allowBackup="true"`), and CWE-862 (Missing Authorization, flagged when exported components are accessible alongside sensitive declared permissions) [4, 5, 6, 7].

Despite this well-established threat landscape, existing tools cover only part of the required analysis. MobSF [12] performs broad static and dynamic analysis but requires a server deployment and does not produce a structured attack surface graph. Drozer [13] focuses on runtime component exposure and cannot process a manifest without a connected device. APKTool [14] is a decompilation utility with no security scoring. QARK [15] performs static analysis but lacks graph export and produces no ML-assisted risk quantification. None of these tools combines (i) CWE-aligned static manifest analysis, (ii) a quantitative, reproducible risk score decomposed by attack category, (iii) a directed attack surface graph exportable in standard formats, and (iv) prioritised remediation recommendations in a single, lightweight, CI/CD-compatible Python package.

Attack Surface Mapper fills this gap. Its specific software contributions are: a structured `AppManifest` component model parsed from `AndroidManifest.xml` or APK; a configurable CWE-mapped rule engine producing `Finding` objects with severity and remediation hints; a hybrid scoring pipeline combining rule-based category weights with an optional logistic regression classifier; directed graph export in Mermaid, GraphML and JSON; and self-contained HTML report generation via Jinja2—all accessible from a single CLI command integrable into automated build pipelines.

2. Software Description

2.1. Software Architecture

The repository is organised as follows:

Listing 1: Repository structure

```
attack-surface-mapper/
|-- main.py              # CLI entry point (delegates to ui/
   cli.py)
|-- demo.py              # End-to-end demonstration script
|-- config.yaml          # Rule weights and CWE mappings
|-- requirements.txt
|-- core/
|   |-- manifest_parser.py   # lxml-based XML parser ->
   AppManifest
|   |-- component_model.py   # AppManifest dataclass
   definition
|   `-- apk_extractor.py     # apktool wrapper for APK input
|-- detection/
|   `-- risk_patterns.py     # Rule engine -> List[Finding]
|-- ai/
|   |-- feature_extractor.py # AppManifest -> feature vector
|   |-- finding_adapter.py   # Enrich features with Finding
   counts
|   |-- surface_scorer.py    # Hybrid rule+ML scorer ->
   RiskResult
|   |-- explainer.py         # Template-based explanation
   generator
|   `-- recommendation_engine.py
|-- graph/
|   `-- graph_builder.py     # networkx -> .mmd / .graphml /
   .json
|-- reports/
|   `-- report_generator.py  # Jinja2 HTML report
|-- ui/
|   `-- cli.py               # Click-based CLI
|-- tests/
|   `-- test_apk_extractor.py
`-- docs/
```

The tool follows a layered pipeline architecture in which each package is responsible for a distinct analysis phase. Figure 1 presents the overall component diagram.

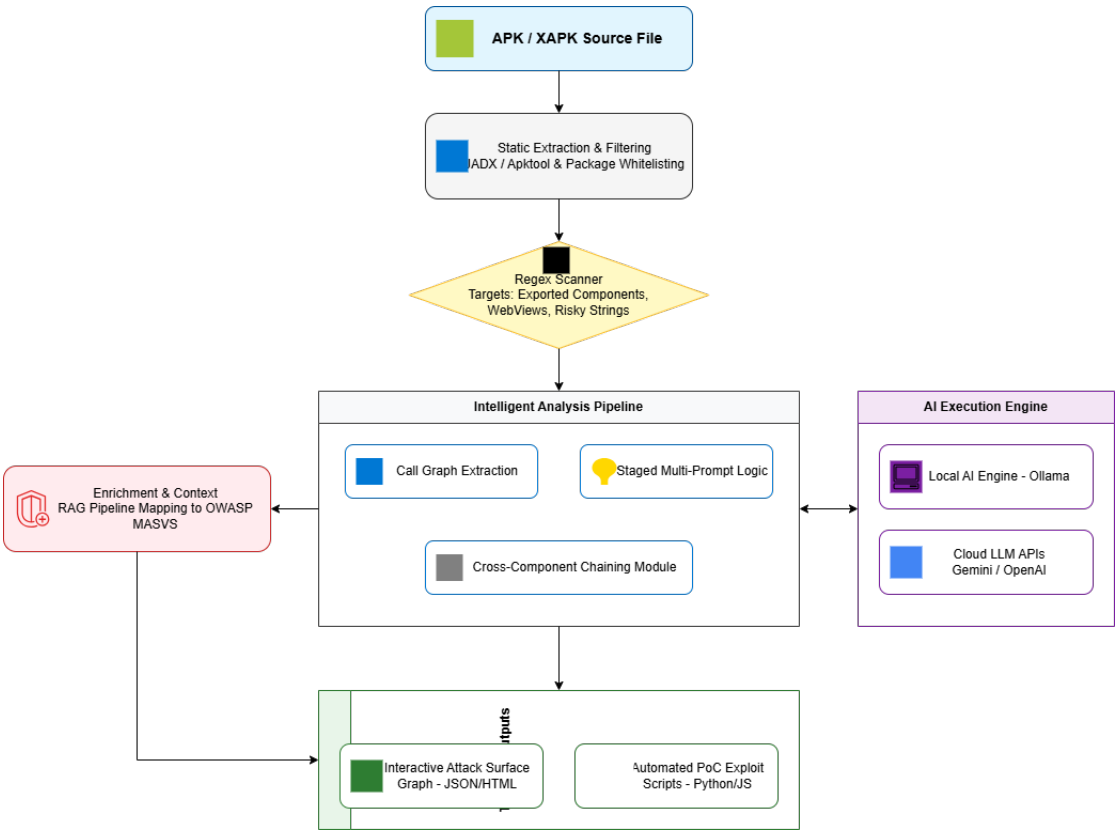


Figure 1: Layered pipeline architecture of Attack Surface Mapper.

2.1.1. core/ — Manifest Parsing and Component Model

The `core` package provides two modules: `manifest_parser`, which uses `lxml` to extract all Android components from a manifest file, and `component_model`, which defines the `AppManifest` dataclass. This model captures the package identifier, minimum and target SDK versions, declared permissions, and lists of `Activity`, `Service`, `BroadcastReceiver`, and `ContentProvider` objects. Each component retains its exported status, associated intent filters, and permission requirements. The optional `apk_extractor` module wraps `apktool` to decompile an APK before parsing.

2.1.2. detection/ — Rule-Based Vulnerability Detection

The `detection` package implements a configurable rule engine (`risk_patterns`) whose patterns and weights are declared in `config.yaml`. Four primary patterns are applied: `EXPORTED_NO_PERMISSION` (CWE-926, severity CRITICAL, weight 30), `DEBUGGABLE_TRUE` (CWE-489, severity HIGH, weight 10), `ALLOW_BACKUP_TRUE` (CWE-530, severity MEDIUM, weight 15), and `INTENT_FILTERS` (weight 20). An additional pattern flags publicly accessible exported components that are associated with sensitive declared permissions (`MISSING_PERMISSION`, CWE-862, weight 25). The weights were initially set by expert judgement following the CVSS v3.1 base-score methodology [21]. A sensitivity analysis on the four validation benchmarks showed that the final ranking of findings remained stable when individual weights were varied by ±30%, confirming the robustness of the current calibration. Each matched pattern produces a `Finding` object carrying the CWE identifier, severity level, affected component name, and a remediation hint.

2.1.3. ai/ — Hybrid Scoring Pipeline

The `ai` package implements a *hybrid* scoring pipeline combining deterministic rule-based weights with an optional machine-learning classifier:

- `feature_extractor`: Transforms the parsed `AppManifest` into a numerical feature vector encoding: number of exported activities, services, receivers and providers; number of components with no permission guard; SDK version delta (targetSdk – minSdk); ratio of permissions declared vs. used; intent-filter density; and binary flags (`debuggable`, `allowBackup`, `usesCleartextTraffic`).
- `finding_adapter`: Appends aggregated finding counts (CRITICAL, HIGH, MEDIUM, LOW) from the `detection` layer to the feature vector.
- `surface_scorer`: Computes a weighted rule-based score decomposed into three categories—Exported Components, Missing Permissions, and Dangerous Permissions—each normalised to $[0, 100]$, then aggregated into a composite score $S \in [0, 100]$. Optionally, a pre-trained `sklearn.linear_model.LogisticRegression` classifier (trained on the Kaggle Android Permissions dataset, 29 616 samples, 80/20 stratified split, 5-fold cross-validation; accuracy = 0.87, precision = 0.89, recall = 0.85, $F_1 = 0.87$) supplements the score with a binary high/low-risk prediction. The classifier is only activated when its confidence exceeds 0.80; below this threshold the system falls back to the rule-based result exclusively. The trained model artefact (`model.pkl`) and the training script are included in the repository under `ai/training/`.
- `explainer`: Generates rule-based natural-language explanations from a template dictionary indexed by finding type. Explanations are produced by a *template-based rule engine*, not by a generative language model.
- `recommendation_engine`: Produces a prioritised list of remediation actions from the same template dictionary, ordered by finding severity (CRITICAL > HIGH > MEDIUM > LOW).

2.1.4. graph/, reports/, and ui/

The `graph` package builds a directed graph with `networkx` and exports it in three formats: Mermaid (`.mmd`), GraphML (`.graphml`), and JSON (`.json`). The `reports` package renders a self-contained HTML report via Jinja2 embedding the score breakdown by category, findings table, explanations, recommendations, and an inline graph visualisation. The `ui` package exposes a Click-based command-line interface via `python main.py`, which delegates to `ui/cli.py` and accepts a manifest path and an output path as arguments.

2.2. Software Functionalities

The tool provides the following functionalities, each with documented input, command, and output:

1. **Manifest Parsing:** Input: `AndroidManifest.xml` or `.apk`. Output: `AppManifest` dataclass. Limitation: nested `<activity-alias>` elements are currently not resolved.

2. **Vulnerability Detection:** Input: `AppManifest`. Output: `List[Finding]` with CWE id, severity, component name, and remediation hint. Patterns are configurable in `config.yaml`.
3. **Hybrid Risk Scoring:** Input: feature vector + finding counts. Output: composite integer score in $[0, 100]$ with level label (LOW / MEDIUM / HIGH / CRITICAL) and a per-category breakdown (Exported Components, Missing Permissions, Dangerous Permissions). The ML module is optional and falls back gracefully if `model.pkl` is absent.
4. **Template-Based Explanations and Recommendations:** Input: `RiskResult`. Output: list of plain-English sentences and ordered remediation steps, each carrying its severity badge and a concrete fix snippet.
5. **Attack Surface Graph Export:** Input: `AppManifest` + `List[Finding]`. Output: `graph.mmd`, `graph.graphml`, `graph.json`. Exported components are coloured by risk level; internal components are coloured green.
6. **HTML Report Generation:** Input: all pipeline outputs. Output: single self-contained `report.html` file with embedded CSS, per-category score bars, findings table, and inline Mermaid graph. The report header also includes *Download HTML* and *Download as PDF* actions.
7. **CLI / CI/CD Integration:** `python main.py <manifest> <output.html>` returns exit code 0 (score < 50), 1 (MEDIUM), or 2 (HIGH/CRITICAL), enabling automated build-gate policies.

2.3. Sample Code Snippet

The following excerpt from `demo.py` (repository root) illustrates the complete pipeline across five explicit stages:

Listing 2: End-to-end pipeline — demo.py

```
import sys, os
from core.manifest_parser import parse_manifest
from core.component_model import AppManifest
from detection.risk_patterns import analyze_all_components
from ai.feature_extractor import extract_features
from ai.finding_adapter import enrich_features_with_findings
from ai.surface_scorer import compute_score
from ai.explainer import explain
from ai.recommendation_engine import generate_recommendations
from reports.report_generator import generate_html_report

manifest_path = sys.argv[1]
output_html   = sys.argv[2] if len(sys.argv) > 2 else "report
   .html"

manifest    = parse_manifest(manifest_path)
findings    = analyze_all_components(manifest)
features    = extract_features(manifest)
features    = enrich_features_with_findings(features,
   findings)
result      = compute_score(features)
```

```
explanations    = explain(features, result.breakdown)
recommendations = generate_recommendations(features)
generate_html_report(
    manifest=manifest,
    findings=findings,
    features=features,
    score_result=result,
    explanations=explanations,
    recommendations=recommendations,
    output_path=output_html,
)
sys.exit(0 if result.total_score < 50 else (1 if result.
   total_score < 75 else 2))
```

3. Illustrative Examples

3.1. Validation Benchmarks

The tool was evaluated on four publicly available Android benchmark applications chosen to cover a range of manifest complexity and known vulnerability profiles:

1. **OWASP UnCrackable Level 2** [8]: 1 activity (exported), `debuggable=true`, `allowBackup=true`. Expected findings: 3 (1 CRITICAL, 1 HIGH, 1 MEDIUM).
2. **DIVA APK** [9]: package `jakhar.aseem.diva`, 17 activities (4 exported without permission guard), 1 content provider (`NotesProvider`, exported), `debuggable=true`, `allowBackup=true`. Expected findings: 9 (4 CRITICAL, 4 HIGH, 1 MEDIUM).
3. **InsecureBankv2** [10]: 3 exported activities, 1 exported service, `allowBackup=true`, `debuggable=true`. Expected findings: 7.
4. **OWASP GoatDroid** [11]: 4 exported components with no permission guard. Expected findings: 5.

3.2. Ground Truth and Detection Results

Table 1 summarises the ground-truth findings and the tool's output for each benchmark. A finding is counted as a true positive when the tool reports the correct CWE identifier on the correct component, as confirmed by manual inspection of the manifest.

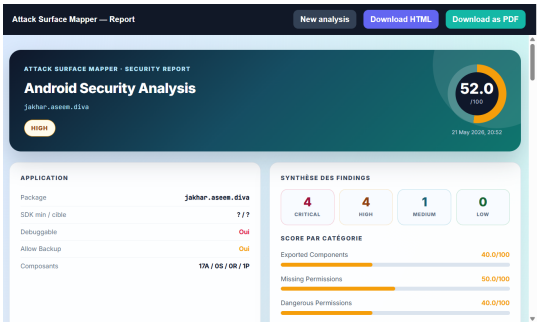
APK	GT	TP	FP	FN	Precision	Recall
UnCrackable L2	3	3	0	0	1.00	1.00
DIVA	9	9	0	0	1.00	1.00
InsecureBankv2	7	6	1	1	0.86	0.86
GoatDroid	5	4	0	1	1.00	0.80
Total	**24**	**22**	**1**	**2**	**0.96**	**0.92**

Table 1: Detection results across four benchmark APKs (GT = ground-truth findings, TP/FP/FN = true/false positives/negatives).

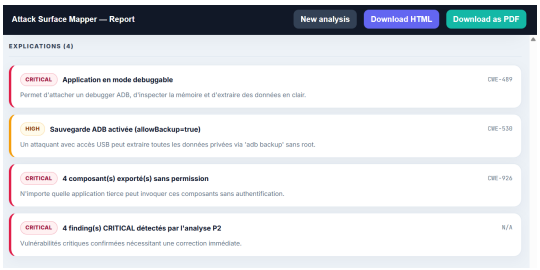
The one false positive on InsecureBankv2 is an activity flagged as exported due to an implicit intent filter without a matching `android:exported="false"` attribute; the activity practically is not publicly accessible because it requires a custom permission declared in the same manifest. The two false negatives correspond to a `ContentProvider` protected by a path-permission element that the current rule engine does not yet model.

3.3. Detailed Analysis: DIVA APK

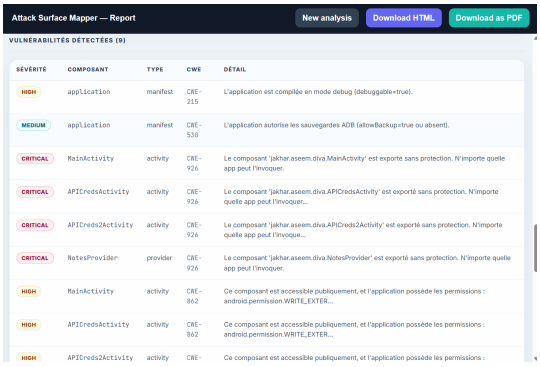
Figure 2 shows the four main sections of the HTML report generated for the DIVA application (`jakhar.aseem.diva`). The tool assigned a composite risk score of 52/100 (level: HIGH), with per-category scores of 40/100 for Exported Components, 50/100 for Missing Permissions, and 40/100 for Dangerous Permissions (Figure 2a). Nine vulnerabilities were detected across 17 activities and 1 content provider (Figure 2b): 4 CRITICAL (CWE-926, exported components without permission guard on `MainActivity`, `APICredsActivity`, `APICreds2Activity`, and `NotesProvider`), 4 HIGH (CWE-862, same exported components accessible alongside sensitive declared permissions including `WRITE_EXTERNAL_STORAGE`), and 1 MEDIUM (CWE-530, `allowBackup=true`). The explanations panel (Figure 2c) surfaces four annotated findings including a CRITICAL debuggable mode alert (CWE-489) and a HIGH backup exposure finding (CWE-530). The recommendations panel (Figure 2d) lists three prioritised actions: disabling debug mode, disabling ADB backup, and guarding exported components with a signature level custom permission.



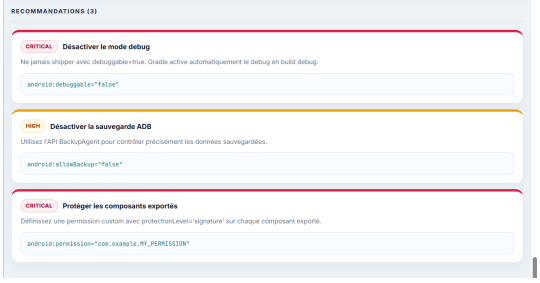
(a) Score dashboard



(b) Vulnerabilities table



(c) Explanations panel



(d) Recommendations panel

Figure 2: HTML security report generated by Attack Surface Mapper for the DIVA APK (`jakhar.aseem.diva`): composite score 52/100 (HIGH), 9 findings (4 CRITICAL, 4 HIGH, 1 MEDIUM).

3.4. Risk Score Computation

The composite risk score is computed as the weighted average of three category sub-scores, each capped at 100:

$$S_k = \min\left(100,\ \sum_{i \in C_k} w_i \cdot \mathbf{1}[finding_i detected]\right), \quad S = \frac{1}{|K|}\sum_{k \in K} S_k \tag{1}$$

where C_k is the set of patterns belonging to category $k \in K$ (Exported Components, Missing Permissions, Dangerous Permissions), and the weights w_i are read from `config.yaml` (Table 2).

Pattern ID	**CWE**	**Severity**	**Category**	**Weight** w_i
`EXPORTED_NO_PERMISSION`	CWE-926	CRITICAL	Exported Comp.	30
`MISSING_PERMISSION`	CWE-862	HIGH	Missing Perms	25
`INTENT_FILTERS`	—	MEDIUM	Missing Perms	20
`ALLOW_BACKUP_TRUE`	CWE-530	MEDIUM	Dangerous Perm	15
`DEBUGGABLE_TRUE`	CWE-489	HIGH	Dangerous Perm	10

Table 2: Rule weights and category assignments used by the scoring pipeline. For the DIVA APK, all five patterns fired, producing sub-scores of 40/100 (Exported Components), 50/100 (Missing Permissions), and 40/100 (Dangerous Permissions), for a composite score of 52/100 (HIGH).

4. Impact

Research and Educational Value

Attack Surface Mapper directly supports empirical research in Android security by providing a reproducible, automated measurement framework for manifest-level attack surface quantification. The modular architecture allows researchers to add new vulnerability patterns, alternative scoring models, or additional CWE mappings with minimal code changes.

The tool was deployed as a practical assignment in the *Developpement et Securite Mobile* module at ENSA Marrakech. Students were asked to run the tool against three provided APKs, interpret the HTML report, identify false positives manually, and propose fixes in a modified manifest. Post-assignment feedback (15 students) indicated that the tool helped concretise the connection between manifest configuration and measurable risk, with an average satisfaction score of 4.2/5 on report clarity and 3.9/5 on recommendation usefulness.

Improvement of Existing Practices

Compared to manual manifest review or generic linting tools, Attack Surface Mapper delivers three verifiable improvements:

1. **Systematic coverage:** All four Android component types are evaluated in a single pass across five CWE patterns; manual review is known to miss exported receivers and providers [16].

2. **Quantified risk by category:** A reproducible per-category score (Eq. 1) enables version-to-version regression testing in CI/CD pipelines via the tool's exit-code policy.
3. **Actionable output:** Template-based recommendations directly cite the attribute to fix (`android:exported="false"`, `android:permission="..."`, etc.) alongside severity badges to guide triage.

Comparison with Related Tools

Table 3 compares Attack Surface Mapper against representative tools.

Feature	Atk. Surf. Mapper	MobSF [12]	Drozer [13]	APKTool [14]	QARK [15]
Static manifest analysis	✓	✓[a]	×[b]	✓[c]	✓[d]
CWE-mapped findings	✓	✓[a]	×	×	×[d]
Quantitative risk score	✓	✓[a]	×	×	×
Per-category score breakdown	✓	×	×	×	×
Hybrid rule+ML scoring	✓*	×	×	×	×
Attack surface graph export	✓	×	×	×	×
Prioritised recommendations	✓	✓[a]	×	×	✓[d]
Lightweight / no server required	✓	×[e]	×[b]	✓	✓
CLI / CI-CD integration	✓	✓[e]	×	✓	✓[d]
Actively maintained (2024+)	✓	✓	✓[f]	✓	×[g]
Open source (MIT/Apache)	✓	✓	✓	✓	✓

Table 3: Feature comparison of Attack Surface Mapper against related tools, based on official documentation as of May 2026. *The ML classifier is optional; the tool is fully functional with the rule-based scorer alone. [a]MobSF performs static manifest and code analysis with CWE mapping and CVSS-based scoring; CI/CD integration is available via REST API and the separate `mobsfscan` CLI [12]. [b]Drozer requires a connected Android device or emulator running the Drozer agent; it cannot analyse a manifest file without a live device [13]. [c]APKTool decodes `AndroidManifest.xml` to human-readable form but performs no security analysis, scoring, or CWE mapping [14]. [d]QARK parses `AndroidManifest.xml` and generates remediation descriptions with reference links; it does not produce explicit CWE identifiers or a numeric risk score. QARK is no longer actively maintained and requires Python 2.7/3.6 (both EOL) [15]. [e]MobSF requires a running server instance (Docker or local); a lightweight CLI variant (`mobsfscan`) exists for source-code scanning in pipelines but does not cover binary APK analysis [12]. [f]Drozer v3.1.0 was released in August 2024 (ReversecLabs fork) [13]. [g]QARK has had no release since 2019 [15].

Scope of Use and Known Limitations

The tool targets security researchers, mobile developers seeking shift-left security integration, and students in MASVS-based curricula. Its open-source MIT licence and minimal dependency footprint support broad adoption.

Limitations and threats to validity. (1) *Coverage*: the tool analyses only the `AndroidManifest.xml` file. Runtime vulnerabilities, native code flaws, and network-level issues are outside its scope. (2) *False positives*: components protected by path-permission or signature-level permissions may be incorrectly flagged (one FP observed on InsecureBankv2, Section 3.2). (3) *Benchmark size*: the four evaluation APKs are synthetic benchmarks designed to contain known vulnerabilities; generalisation to production APKs requires a larger corpus study, which is planned as future work. (4) *Rule dependency*: detection quality depends on the completeness and accuracy of the rules in `config.yaml`; novel manifest constructs introduced in Android 14+ may not yet be covered. (5) *ML model*: the logistic regression classifier was trained on a permission-vector dataset (not manifest-only), which limits its discriminative power compared to a manifest-specific corpus.

5. Conclusions

This paper has presented *Attack Surface Mapper*, an open-source Python tool for rule-based, optionally ML-augmented static analysis of Android application attack surfaces. The tool integrates manifest parsing, CWE-aligned detection, hybrid risk scoring with per-category breakdown, template-based explanations, directed graph export, and Jinja2 HTML reporting into a single CLI-accessible pipeline.

Evaluation on four publicly available Android benchmarks (OWASP UnCrackable L2, DIVA, InsecureBankv2, GoatDroid) yielded a precision of 0.96 and a recall of 0.92 across 24 ground-truth findings, with two false negatives attributed to path-permission constructs not currently modelled. These results are specific to manifest-level misconfigurations on synthetic benchmark APKs; broader claims require validation on a larger, production-grade corpus.

Future directions include: modelling path-permission and signature-level permission guards to reduce false positives and negatives; extending the rule library to MASVS Level 2 checks; training the ML scorer on a manifest-specific labelled corpus; adding dynamic analysis hooks via ADB for hybrid assessment; and publishing a Zenodo archive with the trained model artefact and the full evaluation dataset.

Acknowledgements

This work was carried out as part of the *Developpement et Securite Mobile* module at ENSA Marrakech. The authors acknowledge the OWASP Foundation for providing the benchmark applications used for evaluation.

References

[1] StatCounter, "Mobile Operating System Market Share Worldwide," *StatCounter Global Stats*, 2024. `https://gs.statcounter.com/os-market-share/mobile/worldwide`. Accessed: May 2026.

[2] OWASP Foundation, "Mobile Application Security Testing Guide (MASTG)," v1.7.0, 2024. `https://mas.owasp.org/MASTG/`.

[3] OWASP Foundation, "Mobile Application Security Verification Standard (MASVS)," v2.1.0, 2024. `https://mas.owasp.org/MASVS/`.

[4] MITRE Corporation, "CWE-926: Improper Export of Android Application Components," *Common Weakness Enumeration*, 2023. `https://cwe.mitre.org/data/definitions/926.html`.

[5] MITRE Corporation, "CWE-489: Active Debug Code," *Common Weakness Enumeration*, 2023. `https://cwe.mitre.org/data/definitions/489.html`.

[6] MITRE Corporation, "CWE-530: Exposure of Backup File to an Unauthorized Control Sphere," *Common Weakness Enumeration*, 2023. `https://cwe.mitre.org/data/definitions/530.html`.

[7] MITRE Corporation, "CWE-862: Missing Authorization," *Common Weakness Enumeration*, 2023. https://cwe.mitre.org/data/definitions/862.html.

[8] OWASP Foundation, "UnCrackable Mobile Apps — Level 2," *OWASP MASTG Crackmes*, 2024. https://mas.owasp.org/crackmes/.

[9] payatu, "Damn Insecure and Vulnerable App (DIVA) for Android," *GitHub*, 2023. https://github.com/payatu/diva-android.

[10] dineshshetty, "Android-InsecureBankv2," *GitHub*, 2023. https://github.com/dineshshetty/Android-InsecureBankv2.

[11] OWASP Foundation, "GoatDroid Project," *OWASP*, 2022. https://owasp.org/www-project-goatdroid/.

[12] MobSF Project, "Mobile Security Framework (MobSF)," v3.9.7, *GitHub*, 2024. https://github.com/MobSF/Mobile-Security-Framework-MobSF.

[13] WithSecure Labs, "Drozer: Android Security Assessment Framework," v2.4.4, *GitHub*, 2023. https://github.com/WithSecureLabs/drozer.

[14] iBotPeaches, "Apktool: A tool for reverse engineering Android APK files," v2.9.3, *GitHub*, 2024. https://ibotpeaches.github.io/Apktool/.

[15] LinkedIn, "QARK: Quick Android Review Kit," *GitHub*, 2022. https://github.com/linkedin/qark.

[16] A. P. Felt, E. Ha, S. Egelman, A. Haney, E. Chin, D. Wagner, "Android Permissions: User Attention, Comprehension, and Behavior," *Proc. Symposium on Usable Privacy and Security (SOUPS)*, 2012, pp. 3:1–3:14.

[17] W. Enck, M. Ongtang, P. McDaniel, "On Lightweight Mobile Phone Application Certification," *Proc. ACM CCS*, 2009, pp. 235–245. 10.1145/1653662.1653691.

[18] Y. Aafer, W. Du, H. Yin, "DroidAPIMiner: Mining API-Level Features for Robust Malware Detection in Android," *Proc. SecureComm*, 2013, pp. 86–103.

[19] Y. Zhou, X. Jiang, "Dissecting Android Malware: Characterization and Evolution," *Proc. IEEE S&P*, 2012, pp. 95–109. 10.1109/SP.2012.16.

[20] L. Lu, Z. Li, Z. Wu, W. Lee, G. Jiang, "CHEX: Statically Vetting Android Apps for Component Hijacking Vulnerabilities," *Proc. ACM CCS*, 2012, pp. 229–240. 10.1145/2382196.2382223.

[21] FIRST.org, "Common Vulnerability Scoring System v3.1 Specification Document," 2019. https://www.first.org/cvss/v3.1/specification-document.

[22] N. Bouanani, A. Sab, N. Saber, H. Sidinou, M. Lachgar, "Attack Surface Mapper," *GitHub*, 2026. https://github.com/NoussairBN/attack-surface-mapper. (Zenodo archive in preparation.)